Design and Implementation of a Linux Resource Monitor Using Python, Concurrency, and SQLite Persistence

Julio Cesar Blacio Machuca Ariel Jose Llumiquinga Ñacato
Carrera de Ingeniería de Software
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador

***Abstract*—Monitoring system resources is essential to understanding the current state of an operating system, yet it can remain an abstract concept when examined only through isolated commands. This paper presents the design and implementation of a Linux terminal application developed in Python 3. The application organizes the six monitoring modules according to the Model–View–Controller pattern: CPU, memory and *swap*, processes, connected users, filesystems, and networking. Data are obtained directly from the `/proc` virtual filesystem and, where appropriate, from native commands executed in a controlled manner. To meet operating-systems requirements, the application explicitly creates threads with `threading.Thread` and a child process with `os.fork()`, communicating through a pipe. Complete captures are persisted in SQLite and managed through CRUD operations. Verification performed on Ubuntu under WSL recorded 69 successful tests with no failures or skips in 0.870 s. The result is a traceable educational foundation that integrates observation, concurrency, and persistence without replacing system interfaces with high-level monitoring dependencies.**

***Index Terms*—Linux, resource monitoring, Python, concurrency, SQLite, proc filesystem**

I. Introduction

Resource management makes it possible to relate the activity of processes, memory, storage, and communications to the observable behavior of an operating system. In particular, concepts such as processes, scheduling, memory, and input/output require concrete mechanisms that move from theoretical description to verifiable observation [1]. Linux exposes part of that state through kernel interfaces and user-space commands, making it possible to build focused and transparent tools for educational purposes.

The *Linux Resource Monitor* project addresses this need with an interactive console application intended to observe and record the state of a Linux computer without modifying processes, networking, memory, or filesystems. Its scope combines resource collection, an explicit demonstration of threads and processes, and persistence of complete captures. It is not intended to replace specialized tools or to serve as a performance benchmark.

This work contributes a single terminal application that brings together monitoring across six domains, direct `/proc` reading, native commands, explicit use of `threading.Thread` and `os.fork()`, and CRUD operations over SQLite. Its structure also provides automated evidence so that functional claims can be reviewed on Linux.

The technical objective is not limited to listing values: it preserves the provenance of each datum from its system source to presentation or storage. This required structured contracts among layers, a partial-failure policy for live queries, and a stricter persistence rule: a capture is stored only when it contains all six required modules. This distinction allows a changing system to be observed without confusing an incomplete live view with a complete historical record.

II. Background and Related Work

The `/proc` virtual filesystem publishes information generated by the kernel and running processes; it is therefore a suitable interface for querying counters and descriptors without requiring an intermediate database [2]. This work uses it for CPU, memory, load, and network traffic, while sessions, processes, and mount points are obtained through native utilities that expose these system views.

The two source classes have different properties. Files under `/proc` represent state at the instant of each read, whereas utilities produce a snapshot whose composition may change before it is displayed. Consequently, the design does not require a process count to equal a later execution of `ps`; it requires well-formed records with traceable fields. Likewise, cumulative network counters are retained as bytes and packets and are not interpreted as rates without a measurement interval.

From an operating-systems perspective, separating collection, presentation, and coordination prevents terminal formatting from being mixed with resource access. This decision makes it easier to compare measurements, preserve them as captures, and manage errors by module. It also supports the study of the difference between threads that share a process and a child created with `fork()`, without concealing these mechanisms behind high-level monitoring libraries [1].

III. Methodology

A. Incremental Development

Development was organized over five weeks. The first defined requirements, the MVC architecture, and the relational schema; the second incorporated CPU, memory, and `/proc` reading; the third added disk, networking, users, and processes;

the fourth integrated threads, `fork()`, and CRUD; and the fifth was dedicated to integration, tests, manuals, and evidence. This sequence kept each increment associated with a data source and observable acceptance criteria.

B. Capture Flow

A request for the overall state crosses four stages. First, the controller registers one collector function per module. Second, the coordinator creates one `Thread` object for each function, starts all six, and waits for completion. Third, it consolidates three separate structures: `datos`, `errores`, and `evidencias`; the last retains the module name, thread name, and UTC start and finish instants. Finally, the view can present available data and warn about errors. If the user chooses to record the state, the CRUD controller applies an additional validation before delegating the transaction to the repository.

The separation between querying and persistence is deliberate. During a live query, failure of one source does not discard information obtained by the others. By contrast, `consolidar_captura` verifies that CPU, memory, disk, networking, processes, and users are present and that the error dictionary is empty. If a module is missing, a controlled error is raised and no write operation is opened for an incomplete capture.

C. Sources and Calculations

CPU information is described through `/proc/cpuinfo`, `/proc/loadavg`, and two readings of `/proc/stat`. Utilization is not inferred from load average; it is calculated from the deltas of total and idle time,

$$U_{CPU} = 100 \left(1 - \frac{\Delta T_{idle}}{\Delta T_{total}} \right), \tag{1}$$

where both counters come from the aggregate CPU line. For memory, the distinction between free and available memory is preserved: used memory is `MemTotal - MemAvailable`, while used *swap* is `SwapTotal - SwapFree`. Networking uses `/proc/net/dev` for counters and `ip` for addresses; `ps`, `who`, and `df -P -B1` complement the process, user, and mounted-filesystem data.

D. Data Contracts

Models do not return terminal blocks. CPU and memory produce dictionaries; disk, networking, processes, and users produce lists of dictionaries, with one element per mount, interface, process, or session. Stable keys include `porcentaje_uso`, `mem_disponible_mb`, `punto_montaje`, `usuario_propietario`, `direccion_ip`, and `inicio_sesion`. The controller preserves these structures, while the view adds units, rounds values to two decimal places, and builds ASCII tables.

Each parser accepts text and can therefore be tested without querying the current computer. Networking combines counters from `/proc/net/dev` with one address per interface: it prefers the first IPv4 address and uses IPv6 as a fallback; if no address exists, it retains `None`. Disk data are parsed as bytes from `df -P -B1`; persistence converts them to the GB columns, and historical reads reconstruct bytes before returning the contract to the view. For users, the abbreviated instant reported by `who` is normalized as `YYYY-MM-DD HH:MM`, including year rollover; duration is calculated when `inicio_sesion` is displayed and does not replace the stored value.

E. Concurrency and Safety

Six independent threads, one for each module, are created before the application waits for their completion. Results, errors, and evidence are protected by a lock so that a partial response can be consolidated when a reading fails. The child process starts only after those threads have finished; this order avoids calling `fork()` while other threads are active. The child reads a load metric, communicates a JSON message through a pipe, and is reaped by the parent with `waitpid()`.

The demonstration also checks that no other threads are active before creating the child. Parent and child close the pipe end they do not use; the child terminates with `os._exit()`, and the parent converts the state returned by `waitpid()` into an exit code. A nonzero code or an empty pipe becomes a comprehensible error. The database does not participate in this flow, and no SQLite connection is shared with worker threads or with the child process.

F. Failures and Changing State

Commands are executed with an argument list, captured standard output and error, text mode, a five-second timeout, `LC_ALL=C`, and no `shell=True` [3]. A nonzero return becomes a `RuntimeError`; a missing command, timeout, unavailable `/proc` file, or incomplete content propagates an exception that the worker records under the module name. Thus, the menu receives controlled messages instead of an abrupt termination.

Parsers ignore empty or nonconforming lines when they can continue and reject missing mandatory fields when the result would lose meaning. This criterion matters for tables that change during observation: a process may terminate between two queries, and an interface may have no IP address. The application accepts empty lists and unavailable addresses but does not invent values to complete a capture.

G. Verification Strategy

Unit tests validate parsing functions against sample files so that their results do not depend solely on the current machine. Persistence tests use temporary SQLite databases and cover CRUD operations and cascade deletion. Integrations requiring `fork()` are explicitly limited to Linux. The documented execution was performed on Ubuntu under WSL; therefore, this article does not extend that evidence to other systems.

Verification also covers contracts between layers: the controller preserves partial errors, an incomplete capture is rejected, connections enable foreign keys, update affects only metadata, and deletion removes child rows. Interface tests exercise real dates under a strict format, nonpositive identifiers, explicit confirmation variants, display numbering, deletion

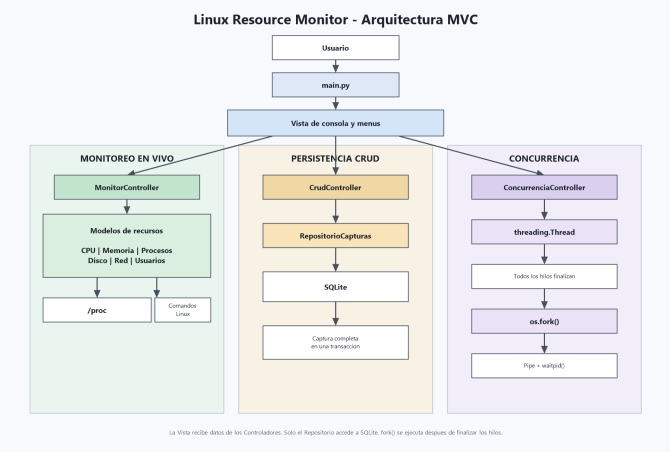

Fig. 1. MVC architecture, concurrent collection, and monitor persistence.

cancellation, and `EXITO`, `ADVERTENCIA`, and `ERROR` messages.

IV. Solution Architecture and Development

A. MVC Separation

Fig. 1 summarizes the organization. The model reads and transforms system data and encapsulates the SQLite repository. The view requests keyboard input and presents menus, tables, and messages. The controller coordinates collection, validation, concurrency, and the CRUD flow. This separation preserves structured data contracts among layers and prevents SQL queries or `/proc` reads in the view.

The `MonitorController` facade supports requesting either one module or the overall state without exposing how collection occurs. `CrudController` receives the concurrent result, requires a complete capture, and translates SQLite failures into an application exception. The repository is the only component that knows SQL statements; consequently, terminal navigation and integrity rules can be tested with doubles without executing Linux commands.

B. Monitoring Modules

Table I relates each module to its primary source and the result delivered to the controller. Command invocations use argument lists, captured output, timeouts, and no `shell=True`; using `subprocess` preserves a clear boundary between the application and system commands [3]. The Python interfaces used for processes and threads correspond to the documented `os` and `threading` APIs [4], [5].

The CPU module separates load from utilization: the three load values come from `/proc/loadavg`, whereas the percentage uses deltas from `/proc/stat`. The count is named logical processors because it comes from `processor` entries in `/proc/cpuinfo`; it is not presented as physical cores. Memory parsing validates five mandatory fields before converting KiB to MiB. Processes, users, and disk each parse one invocation of `ps`, `who`, or `df`, respectively, preserving a snapshot consistent with the invoked source.

TABLE I
Traceability between module, source, and collected data.

Module	Primary source	Collected data
CPU	`/proc/cpuinfo`, `/proc/stat`, `/proc/loadavg`	Logical processors, model, frequency, load, and utilization
Memory	`/proc/meminfo`	Total, free, available, used, and *swap*
Processes	`ps`	PID, name, state, and owner
Users	`who`	Username, terminal, and login start time
Disk	`df -P -B1`	Mount point, bytes, and utilization percentage
Network	`/proc/net/dev`, `ip`	Interface, address, and traffic counters

C. Concurrency and Persistence

Collection starts the six `Thread` objects, joins them before consolidating the result, and retains errors by module. The child process is created through `os.fork()`, sends its data through a `pipe`, and is reaped without leaving orphan processes. This sequence makes the difference between thread coordination and interprocess communication explicit.

Captures are stored in a main table with related CPU, memory, disk, process, network, and user metrics. Each connection enables foreign keys, and every complete capture is written in one transaction, so a required failure rolls back the operation. The relationships use cascade deletion to remove associated metrics when a capture is deleted [6], [7]. CRUD supports creating, listing, and consulting captures, updating their metadata, and deleting them through the repository.

The relational design distinguishes cardinalities. CPU and memory have a one-to-one relationship with `capturas`: their primary key is also a foreign key. Disk, processes, networking, and users have one-to-many relationships and their own keys because a capture may include several elements of each type. The parent table retains date, label, comment, and recording user; update changes only the label and comment, never observed metrics.

Writing opens an independent connection and enters the transaction context before inserting the parent row and the six metric families. If a required key is absent, a contract type is invalid, or SQLite rejects a row, the context rolls back the set and the `finally` block closes the connection. During reading, the repository reconstructs the same contract used by the view: one dictionary for CPU and memory and lists for the one-to-many relationships.

D. Terminal Interface Criteria

The main and history menus use numbered options. Invalid input keeps the loop active, and the optional filter requires a real date in `YYYY-MM-DD` format. Available captures are shown before consulting, updating, or deleting; each row has a sequential `N.` separate from its persistent ID. Existing IDs are not renumbered, and the sequence resets only when history becomes empty. Deletion still requires an explicit `SI`, accepted without case or accent sensitivity. Long listings are initially

TABLE II
VERIFIED RESULTS FROM THE DOCUMENTED EXECUTION.

Validation	Result	Status
Windows suite	69 tests; 2 skips; 0.261 s	0 failures
Ubuntu WSL suite	69 tests; 0 skips; 0.870 s	0 failures
Overall state	6 modules	0 errors
SQLite CRUD	Create/list/update/delete	Correct
Concurrency	6 threads + 1 child	Exit status 0

limited to 20 rows and state how many records are displayed relative to the total.

Output does not depend on color. Percentages are accompanied by `NORMAL`, `ADVERTENCIA`, or `CRITICO`; values include units and two decimal places, and the overall state includes an update timestamp. Missing values are expressed as *No disponible* or with specific empty-list messages. The interface therefore remains understandable in a plain-text terminal and through keyboard operation.

V. RESULTS AND VALIDATION

Table II summarizes the final validation and manual evidence obtained on July 18, 2026, on Ubuntu. The suites were run with `python -m unittest discover -s tests` on Windows and with `python3` on Ubuntu under WSL; the other records exercised the monitoring controller, a temporary SQLite repository, and the concurrency demonstration. The documented durations belong to those functional runs and are not presented as performance measurements.

The overall-state evidence recorded all six modules with an empty error dictionary; Fig. 2 shows CPU, memory, disk, and networking in the Ubuntu terminal. In the persistence test, a capture was created, listed, had its metadata updated, and was deleted. The concurrency demonstration produced six threads without errors, followed by a child with a PID different from the parent's and an exit code of zero, as shown in Fig. 4.

The suite combines deterministic tests and integration checks. Parsers for CPU, memory, disk, networking, processes, and users consume fixture files; the repository is created in a temporary directory and verifies both the reconstructed detail and cascade behavior; controllers are exercised with replaceable collectors and repositories; and Linux cases validate the integrated flow and a real `fork()`. This composition reduces dependence on ephemeral values without removing the final check on the target system.

Manual Ubuntu validation produced five corrected findings. First, abbreviated `who` timestamps were normalized before calculating live duration. Second, historical disk reads convert stored GB values back to the byte contract expected by the view. Third, the filter rejects malformed formats and invalid calendar dates instead of performing an empty search. Fourth, `N.` numbers the display without replacing the stable ID; the sequence returns to one only after the last capture is deleted. Fifth, consult, update, and delete display history first, while explicit `SI` confirmation remains mandatory but ignores case and the accent. Fig. 3 retains pre-correction diagnostic evidence,

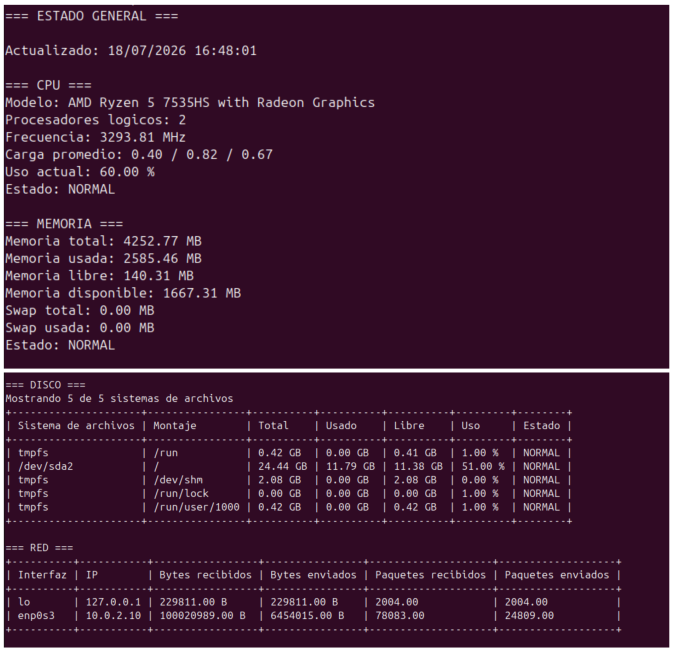

Fig. 2. Overall state observed on Ubuntu: CPU, memory, filesystems, and networking.

including the `ID` header and ID 2 appearing after deletion; it is not proof of the corrected interface. That behavior is verified by regression tests and the complete WSL suite.

Acceptance criteria were interpreted according to the nature of each datum. Key sets and formulas are required for CPU and memory, command records must be parseable, networking retains one primary address or explicit absence, the live view continues after partial failures, and history writes remain atomic. Exact equality is not required between counts taken at different instants, and observed execution times are not generalized.

VI. CONCLUSIONS

The monitor integrates the functional requirements for observing CPU, memory, processes, users, disk, and networking in a Linux terminal application. MVC separation keeps resource reading, coordination, and presentation distinct; in turn, the repository centralizes SQLite persistence and CRUD operations.

The demonstration of six threads and one child illustrates a relevant ordering decision: threads finish before `fork()` begins, and the parent retrieves the child through a pipe and `waitpid()`. The transaction for every complete capture and cascading foreign keys support the consistency of stored data. The five corrected incidents also show the importance of validating contracts among collection, persistence, and presentation. The documented verification supports the behavior described without attributing performance properties to it.

As limitations, the readings depend on the available Linux interfaces and represent changing system states. Future work may study portability in the collection layer or longer-duration

Registro

```
=== HISTORIAL DE CAPTURAS ===
[1] Registrar captura
[2] Listar capturas
[3] Consultar detalle
[4] Actualizar metadatos
[5] Eliminar captura
[0] Volver
Seleccione una opcion: 1
Etiqueta opcional:
Comentario opcional:
EXITO: captura 1 registrada.
```

Eliminacion

```
=== HISTORIAL DE CAPTURAS ===
[1] Registrar captura
[2] Listar capturas
[3] Consultar detalle
[4] Actualizar metadatos
[5] Eliminar captura
[0] Volver
Seleccione una opcion: 5
Identificador de captura: 1
Escriba SI para confirmar la eliminacion: SI
EXITO: captura eliminada.
```

Actualizacion

```
=== HISTORIAL DE CAPTURAS ===
[1] Registrar captura
[2] Listar capturas
[3] Consultar detalle
[4] Actualizar metadatos
[5] Eliminar captura
[0] Volver
Seleccione una opcion: 4
Identificador de captura: 2
Nueva etiqueta opcional: Detalle completo
Nuevo comentario opcional: datos guardados correctamente
EXITO: metadatos actualizados.
```

Listado final

```
=== HISTORIAL DE CAPTURAS ===
[1] Registrar captura
[2] Listar capturas
[3] Consultar detalle
[4] Actualizar metadatos
[5] Eliminar captura
[0] Volver
Seleccione una opcion: 2
Fecha YYYY-MM-DD opcional:
ID | FECHA Y HORA | ETIQUETA
2 | 2026-07-18 17:03:41 | Detalle completo
```

Fig. 3. Pre-correction CRUD diagnostic evidence recorded during Ubuntu validation.

```
=== MENU PRINCIPAL ===
[1] Estado general
[2] CPU
[3] Memoria y swap
[4] Procesos
[5] Disco
[6] Red
[7] Usuarios conectados
[8] Historial y CRUD
[9] Demostracion de hilos y fork
[0] Salir
Seleccione una opcion: 9
=== DEMOSTRACION DE CONCURRENCIA ===
Hilos completados: 6
Errores de hilos: 0
PID padre: 6568
PID hijo: 6878
Estado de salida: 0
Presione Enter para volver al menu principal:
```

Fig. 4. Ubuntu evidence of six threads without errors and a child process exiting with status zero.

samples while preserving the architectural separation and without claiming that these extensions form part of the current version.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. John Wiley & Sons, 2019.

[2] The Linux Kernel Documentation, "The /proc filesystem," https://docs.kernel.org/filesystems/proc.html, accessed: Jul. 17, 2026.

[3] Python Software Foundation, "subprocess — subprocess management," https://docs.python.org/3/library/subprocess.html, accessed: Jul. 17, 2026.

[4] ——, "os — miscellaneous operating system interfaces," https://docs.python.org/3/library/os.html, accessed: Jul. 17, 2026.

[5] ——, "threading — thread-based parallelism," https://docs.python.org/3/library/threading.html, accessed: Jul. 17, 2026.

[6] SQLite Project, "Sqlite foreign key support," https://www.sqlite.org/foreignkeys.html, accessed: Jul. 17, 2026.

[7] ——, "Transaction," https://www.sqlite.org/lang_transaction.html, accessed: Jul. 17, 2026.